VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum, Karnataka -590014.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

R VANDANAA (1BF24CS237)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by N Maanya Sree (1BF24CS193), who is bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of an OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Sandhya A Kulkarni
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index

Lab 1

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin

a) FCFS

```
#include<stdio.h>
struct process{
   int pid, at, bt, ct, tat, wt, rt; };
int main(){
   int n,i,j;
   float avg_wt=0,avg_tat=0,avg_rt=0;
   struct process p[20],temp;
   printf("enter number of processes: ");
   scanf("%d",&n);
   for(i=0;i<n;i++){
      p[i].pid=i+1;
      printf("enter at and bt for P%d: ",p[i].pid);
      scanf("%d %d" , &p[i].at,&p[i].bt);
   }
   for(i = 0;i<n-1;i++){
      for(j=i+1;j<n;j++){
         if(p[i].at>p[j].at){
            temp=p[i];
            p[i]=p[j];
            p[j]=temp;
         }
      }
   }
    p[0].ct = p[0].at + p[0].bt;
   for(i = 1; i < n; i++){
      if(p[i].at > p[i-1].ct)
         p[i].ct = p[i].at + p[i].bt;
```

```
        else
            p[i].ct = p[i-1].ct + p[i].bt;
}

    for(i = 0; i < n; i++){
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;
        p[i].rt = p[i].wt;

        avg_wt += p[i].wt;
        avg_tat += p[i].tat;
        avg_rt += p[i].rt;
    }
    printf("\npid\tat\tbt\tct\ttat\twt\trt\n");

    for(i = 0; i < n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
              p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
    }

    printf("\naverage tat= %.2f", avg_tat/n);
    printf("\naverage wt = %.2f", avg_wt/n);
    printf("\naverage rt = %.2f\n", avg_rt/n);

    return 0;
}
```

OUTPUT:

```
enter number of processes: 4
enter at and bt for P1: 0
8
enter at and bt for P2: 1
4
enter at and bt for P3: 2
9
enter at and bt for P4: 3
2

pid     at      bt      ct      tat     wt      rt
P1      0       8       8       8       0       0
P2      1       4       12      11      7       7
P3      2       9       21      19      10      10
P4      3       2       23      20      18      18

average tat= 14.50
average wt = 8.75
average rt = 8.75

Process returned 0 (0x0)   execution time : 33.966 s
Press any key to continue.
```

b) SJF non pre-emptive

```
#include<stdio.h>
struct process {
    int pid, at, bt, ct, tat, wt, rt;};
int main () {
    int n,i,j;
    float avg_wt=0, avg_tat=0, avg_rt=0;
    struct process p [20], temp;

    printf ("enter number of processes: ");
    scanf ("%d",&n);

    for(i=0; i<n;i++){
        p[i].pid=i+1;
        printf("enter at and bt for P%d: ",p[i].pid);
```

```
    scanf("%d %d" , &p[i].at,&p[i].bt);
}
for (i = 0;i<n-1;i++){
    for (j=i+1; j<n; j++) {
        if(p[i].at>p[j].at) {
            temp=p[i];
            p[i]=p[j];
            p[j]=temp;
        }
    }
}
 p [0].ct = p[0].at + p[0].bt;
for (i = 1; i < n; i++) {
    if(p[i].at > p[i-1]. ct)
        p[i]. ct = p[i].at + p[i].bt;
    else
        p[i].ct = p[i-1].ct + p[i].bt;
}

for(i = 0; i < n; i++){
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    p[i].rt = p[i].wt;

    avg_wt += p[i].wt;
    avg_tat += p[i].tat;
    avg_rt += p[i].rt;
}
printf("\npid\tat\tbt\tct\ttat\twt\trt\n");

for(i = 0; i < n; i++){
```

```
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
                p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
    }

    printf("\naverage tat= %.2f", avg_tat/n);
    printf("\naverage wt = %.2f", avg_wt/n);
    printf("\naverage rt = %.2f\n", avg_rt/n);

    return 0;
}
```

OUTPUT:

```
enter number of processes: 4
enter at and bt for P1: 0
8
enter at and bt for P2: 1
4
enter at and bt for P3: 2
9
enter at and bt for P4: 3
2

pid     at      bt      ct      tat     wt      rt
P1      0       8       8       8       0       0
P2      1       4       12      11      7       7
P3      2       9       21      19      10      10
P4      3       2       23      20      18      18

average tat= 14.50
average wt = 8.75
average rt = 8.75

Process returned 0 (0x0)   execution time : 17.834 s
Press any key to continue.
```

SJF pre-emptive

```
#include <stdio.h>
struct process {
   int pid, at, bt, rt_bt,  ct, tat, wt, rt, st, completed;
};

int main () {
   int n,i,time=0,completed=0,idx;
   float avg_tat=0, avg_wt=0, avg_rt=0;
   struct process p [20];

   printf("Enter num of processes: ");
   scanf("%d",&n);

   for (i=0; i<n;i++){
      p[i].pid=i+1;
      printf("Enter AT and BT for process %d: ",p[i].pid);
      scanf("%d %d",&p[i].at,&p[i].bt);

      p[i].rt_bt=p[i].bt;
      p[i]. completed=0;
      p[i].st=-1;
   }
   while(completed<n) {
      int min_bt=9999;
      idx=-1;
      for(i=0;i<n;i++){
         if(p[i].at<=time && p[i]. completed==0) {
            if(p[i].rt_bt < min_bt && p[i].rt_bt>0){
               min_bt=p[i].rt_bt;
               idx=i;
```

```
            }
        }
    }
    if(idx!=-1){
        if(p[idx].st==-1){
            p[idx].st=time;
            p[idx].rt=p[idx].st-p[idx].at;
        }
        p[idx].rt_bt--;
        time++;
        if(p[idx].rt_bt==0){
            p[idx].ct=time;
            p[idx].tat=p[idx].ct-p[idx].at;
            p[idx].wt=p[idx].tat-p[idx].bt;

            avg_tat+=p[idx].tat;
            avg_wt+=p[idx].wt;
            avg_rt+=p[idx].rt;

            p[idx].completed=1;
            completed++;
        }
    }
    else{
        time++;
    }
}

printf("\nPID\tAT\tBT\tST\tCT\tTAT\tWT\tRT\n");
for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
        p[i].pid,p[i].at,p[i].bt,p[i].st,p[i].ct,p[i].tat,p[i].wt,p[i].rt);
    }

    avg_tat/=n;
    avg_wt/=n;
    avg_rt/=n;

    printf("\nAverage TAT = %.2f",avg_tat);
    printf("\nAverage WT = %.2f",avg_wt);
    printf("\nAverage RT = %.2f\n",avg_rt);

    return 0;
}
```

OUTPUT:

```
Enter num of processes: 4
Enter AT and BT for process 1: 0
8
Enter AT and BT for process 2: 1
4
Enter AT and BT for process 3: 2
9
Enter AT and BT for process 4: 3
2

PID     AT      BT      ST      CT      TAT     WT      RT
P1      0       8       0       14      14      6       0
P2      1       4       1       5       4       0       0
P3      2       9       14      23      21      12      12
P4      3       2       5       7       4       2       2

Average TAT = 10.75
Average WT = 5.00
Average RT = 3.50

Process returned 0 (0x0)   execution time : 13.057 s
Press any key to continue.
```

c) Priority non-pre-emptive

```
#include<stdio.h>
struct process{
   int pid, at, bt, pr, ct, tat, wt, rt, completed;
};

int main(){
   int n,i,time=0,count=0,highest;
   float avg_wt=0,avg_tat=0,avg_rt=0;
   struct process p[20];
   printf("Enter number of processes: ");
   scanf("%d",&n);
   for(i=0;i<n;i++){
      p[i].pid=i+1;
      printf("Enter AT BT Priority for P%d: ",p[i].pid);
      scanf("%d%d%d",&p[i].at,&p[i].bt,&p[i].pr);
      p[i].completed=0;
   }
   while(count<n){
      highest=-1;
      for(i=0;i<n;i++){
         if(p[i].at<=time && p[i].completed==0){
            if(highest==-1 || p[i].pr < p[highest].pr){
               highest=i;
            }
         }
      }

      if(highest==-1){
         time++;
      }
```

```
        else{
            p[highest].rt = time - p[highest].at;
            time += p[highest].bt;

            p[highest].ct = time;
            p[highest].tat = p[highest].ct - p[highest].at;
            p[highest].wt = p[highest].tat - p[highest].bt;

            avg_wt += p[highest].wt;
            avg_tat += p[highest].tat;
            avg_rt += p[highest].rt;

            p[highest].completed=1;
            count++;
        }
    }
    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");

    for(i=0;i<n;i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].pr, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
    }

    printf("\nAverage TAT = %.2f",avg_tat/n);
    printf("\nAverage WT  = %.2f",avg_wt/n);
    printf("\nAverage RT  = %.2f\n",avg_rt/n);

    return 0;
}
```

OUTPUT:

```
Enter number of processes: 4
Enter AT BT Priority for P1: 0
8
3
Enter AT BT Priority for P2: 1
4
1
Enter AT BT Priority for P3: 2
9
4
Enter AT BT Priority for P4: 3
2
2

PID     AT      BT      PR      CT      TAT     WT      RT
P1      0       8       3       8       8       0       0
P2      1       4       1       12      11      7       7
P3      2       9       4       23      21      12      12
P4      3       2       2       14      11      9       9

Average TAT = 12.75
Average WT  = 7.00
Average RT  = 7.00

Process returned 0 (0x0)   execution time : 246.756 s
Press any key to continue.
```

Priority pre-emptive

```
#include<stdio.h>
struct process{
    int pid, at, bt, pr, ct, tat, wt, rt, rbt, started;
};

int main(){
    int n,i,time=0,completed=0,highest;
    float avg_wt=0,avg_tat=0,avg_rt=0;
    struct process p[20];
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        p[i].pid=i+1;
```

```
    printf("Enter AT BT Priority for P%d: ",p[i].pid);
    scanf("%d%d%d",&p[i].at,&p[i].bt,&p[i].pr);
    p[i].rbt=p[i].bt;
    p[i].started=0;
}
while(completed<n){
    highest=-1;
    for(i=0;i<n;i++){
        if(p[i].at<=time && p[i].rbt>0){
            if(highest==-1 || p[i].pr < p[highest].pr){
                highest=i;
            }
        }
    }
    if(highest==-1){
        time++;
    }
    else{
        if(p[highest].started==0){
            p[highest].rt = time - p[highest].at;
            p[highest].started=1;
        }
        p[highest].rbt--;
        time++;
        if(p[highest].rbt==0){
            p[highest].ct=time;
            p[highest].tat=p[highest].ct-p[highest].at;
            p[highest].wt=p[highest].tat-p[highest].bt;

            avg_wt+=p[highest].wt;
            avg_tat+=p[highest].tat;
```

```
            avg_rt+=p[highest].rt;
            completed++;
        }
    }
}
printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");
for(i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    p[i].pid, p[i].at, p[i].bt, p[i].pr, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
}

printf("\nAverage TAT = %.2f",avg_tat/n);
printf("\nAverage WT  = %.2f",avg_wt/n);
printf("\nAverage RT  = %.2f\n",avg_rt/n);

return 0;
}
```

OUTPUT:

```
Enter number of processes: 4
Enter AT BT Priority for P1: 0
8
3
Enter AT BT Priority for P2: 1
4
1
Enter AT BT Priority for P3: 2
9
4
Enter AT BT Priority for P4: 3
2
2

PID     AT      BT      PR      CT      TAT     WT      RT
P1      0       8       3       14      14      6       0
P2      1       4       1       5       4       0       0
P3      2       9       4       23      21      12      12
P4      3       2       2       7       4       2       2

Average TAT = 10.75
Average WT  = 5.00
Average RT  = 3.50

Process returned 0 (0x0)   execution time : 31.299 s
Press any key to continue.
```

d) Round robin

```
#include<stdio.h>
struct process{
   int pid, at, bt, ct, tat, wt, rt;
};

int main(){
   int n, i, time = 0, remain, tq, done;
   float avg_wt=0, avg_tat=0;
   struct process p[20];
   printf("enter number of processes: ");
   scanf("%d",&n);
   for(i=0;i<n;i++){
      p[i].pid = i+1;
      printf("enter at and bt for P%d: ",p[i].pid);
      scanf("%d %d",&p[i].at,&p[i].bt);
      p[i].rt = p[i].bt;
   }
   printf("enter time quantum: ");
   scanf("%d",&tq);
   remain = n;
   while(remain != 0){
      done = 1;
      for(i = 0; i < n; i++){
         if(p[i].rt > 0 && p[i].at <= time){
            done = 0;
            if(p[i].rt > tq){
               time += tq;
               p[i].rt -= tq;
            }
```

```
            else{
                time += p[i].rt;
                p[i].rt = 0;
                p[i].ct = time;
                remain--;
            }
        }
    }
    if(done == 1){
        time++;
    }
}
for(i = 0; i < n; i++){
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    avg_wt += p[i].wt;
    avg_tat += p[i].tat;
}
printf("\npid\tat\tbt\tct\ttat\twt\n");
for(i = 0; i < n; i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
          p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}
printf("\naverage tat = %.2f", avg_tat/n);
printf("\naverage wt  = %.2f\n", avg_wt/n);

return 0;
}
```

OUTPUT:

```
enter number of processes: 4
enter at and bt for P1: 0
7
enter at and bt for P2: 2
4
enter at and bt for P3: 4
9
enter at and bt for P4: 5
5
enter time quantum: 2

pid     at      bt      ct      tat     wt
P1      0       7       22      22      15
P2      2       4       12      10      6
P3      4       9       25      21      12
P4      5       5       21      16      11

average tat = 17.25
average wt  = 11.00
```

```
enter number of processes: 4
enter at and bt for P1: 0
7
enter at and bt for P2: 2
4
enter at and bt for P3: 4
9
enter at and bt for P4: 5
5
enter time quantum: 4

pid     at      bt      ct      tat     wt
P1      0       7       19      19      12
P2      2       4       8       6       2
P3      4       9       25      21      12
P4      5       5       24      19      14

average tat = 16.25
average wt  = 10.00

Process returned 0 (0x0)   execution time : 30.179 s
Press any key to continue.
```

LAB 2

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include<stdio.h>
struct process{
   int pid, at, bt, ct, tat, wt, rt;
   int type;
};


int main(){
   int n, i, time = 0, remain, done;
   float avg_wt = 0, avg_tat = 0;
   struct process p[20];
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for(i = 0; i < n; i++){
      p[i].pid = i + 1;
      printf("Enter arrival time, burst time, and type (0 = System / 1 = User) for P%d: ", p[i].pid);
      scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].type);
      p[i].rt = p[i].bt;
   }
   remain = n;
   while(remain != 0){
      done = 1;
      for(i = 0; i < n; i++){
         if(p[i].rt > 0 && p[i].at <= time && p[i].type == 0){
            done = 0;
```

```
            time++;
            p[i].rt--;
            if(p[i].rt == 0){
                p[i].ct = time;
                remain--;
            }
            break;
        }
    }
    if(done == 1){
        for(i = 0; i < n; i++){
            if(p[i].rt > 0 && p[i].at <= time && p[i].type == 1){
                done = 0;
                time++;
                p[i].rt--;
                if(p[i].rt == 0){
                    p[i].ct = time;
                    remain--;
                }
                break;
            }
        }
    }
    if(done == 1){
        time++;
    }
}
for(i = 0; i < n; i++){
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    avg_wt += p[i].wt;
```

```
        avg_tat += p[i].tat;
    }
    printf("\nPID\tType\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++){
        printf("P%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid,
               (p[i].type == 0) ? "System" : "User",
               p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }
    printf("\nAverage Turnaround Time: %.2f", avg_tat/n);
    printf("\nAverage Waiting Time: %.2f\n", avg_wt/n);
    return 0;
}
```

OUTPUT:

```
Enter number of processes: 4
Enter arrival time, burst time, and type (0 = System / 1 = User) for P1:
0
5
0
Enter arrival time, burst time, and type (0 = System / 1 = User) for P2:
2
3
1
Enter arrival time, burst time, and type (0 = System / 1 = User) for P3:
1
4
0
Enter arrival time, burst time, and type (0 = System / 1 = User) for P4:
3
2
1

PID     Type    AT      BT      CT      TAT     WT
P1      System  0       5       5       5       0
P2      User    2       3       12      10      7
P3      System  1       4       9       8       4
P4      User    3       2       14      11      9

Average Turnaround Time: 8.50
Average Waiting Time: 5.00
```

LAB 3

Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define MAX 10
typedef struct {
   int id, burst, deadline, period, remaining, weight;
} Process;

void sortEDF(Process p[], int n) {
   for(int i=0;i<n-1;i++) {
      for(int j=i+1;j<n;j++) {
         if(p[i].deadline > p[j].deadline) {
            Process temp = p[i];
            p[i] = p[j];
            p[j] = temp;
         }
      }
   }
}
void sortRMS(Process p[], int n) {
   for(int i=0;i<n-1;i++) {
      for(int j=i+1;j<n;j++) {
         if(p[i].period > p[j].period) {
            Process temp = p[i];
            p[i] = p[j];
            p[j] = temp;
         }
```

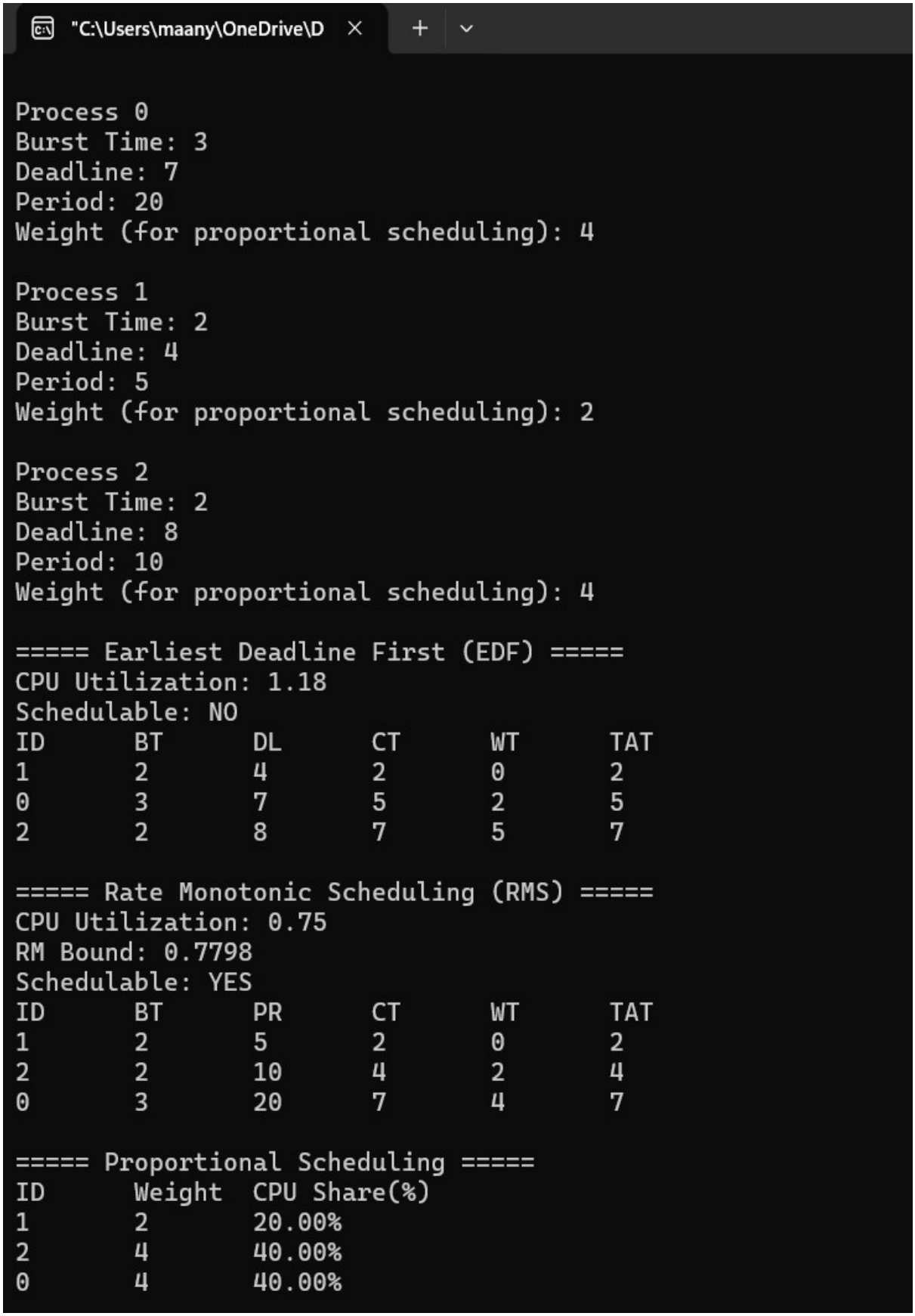
```
        }
    }
}
void EDF(Process p[], int n) {
    float utilization = 0;
    for(int i=0;i<n;i++)
        utilization += (float)p[i].burst / p[i].deadline;
    sortEDF(p, n);
    printf("\n===== Earliest Deadline First (EDF) =====\n");
    printf("CPU Utilization: %.2f\n", utilization);
    printf("Schedulable: %s\n", (utilization <= 1.0) ? "YES" : "NO");
    int time = 0;
    printf("ID\tBT\tDL\tCT\tWT\tTAT\n");
    for(int i=0;i<n;i++) {
        int ct = time + p[i].burst;
        int tat = ct;
        int wt = tat - p[i].burst;
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
              p[i].id, p[i].burst, p[i].deadline, ct, wt, tat);
        time = ct;
    }
}
void RMS(Process p[], int n) {
    float utilization = 0;
    for(int i=0;i<n;i++)
        utilization += (float)p[i].burst / p[i].period;
    float bound = n * (pow(2, 1.0/n) - 1);
    sortRMS(p, n);
    printf("\n===== Rate Monotonic Scheduling (RMS) =====\n");
    printf("CPU Utilization: %.2f\n", utilization);
    printf("RM Bound: %.4f\n", bound);
```

```
    printf("Schedulable: %s\n", (utilization <= bound) ? "YES" : "NO");
    int time = 0;
    printf("ID\tBT\tPR\tCT\tWT\tTAT\n");
    for(int i=0;i<n;i++) {
        int ct = time + p[i].burst;
        int tat = ct;
        int wt = tat - p[i].burst;
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
              p[i].id, p[i].burst, p[i].period, ct, wt, tat);
        time = ct;
    }
}
void Proportional(Process p[], int n) {
    int total_weight = 0;
    for(int i=0;i<n;i++)
        total_weight += p[i].weight;
    printf("\n===== Proportional Scheduling =====\n");
    printf("ID\tWeight\tCPU Share(%%)\n");
    for(int i=0;i<n;i++) {
        float share = ((float)p[i].weight / total_weight) * 100;
        printf("%d\t%d\t%.2f%%\n", p[i].id, p[i].weight, share);
    }
}
int main() {
    int n;
    Process p[MAX];
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(int i=0;i<n;i++) {
        p[i].id = i;
        printf("\nProcess %d\n", i);
```

```
        printf("Burst Time: ");
        scanf("%d", &p[i].burst);
        printf("Deadline: ");
        scanf("%d", &p[i].deadline);
        printf("Period: ");
        scanf("%d", &p[i].period);
        printf("Weight (for proportional scheduling): ");
        scanf("%d", &p[i].weight);
    }
    EDF(p, n);
    RMS(p, n);
    Proportional(p, n);
    return 0;
}
```

OUTPUT:

```
Enter number of processes: 3

Enter process details:

Process 0:
Burst Time: 2
Deadline (for EDF): 5
Period (for RMS): 5

Process 1:
Burst Time: 2
Deadline (for EDF): 7
Period (for RMS): 7

Process 2:
Burst Time: 3
Deadline (for EDF): 8
Period (for RMS): 8

===== Earliest Deadline First (EDF) Scheduling =====
CPU Utilization: 1.06
ID      BF      Deadline        CT      WT      TAT
0       2       5               2       0       2
1       2       7               4       2       4
2       3       8               7       4       7

===== Rate Monotonic Scheduling (RMS) =====
CPU Utilization: 1.06
RM Bound: 0.7798
ID      BF      Period  CT      WT      TAT
0       2       5       2       0       2
1       2       7       4       2       4
2       3       8       7       4       7
```


"C:\Users\maany\OneDrive\D
Process 0
Burst Time: 3
Deadline: 7
Period: 20
Weight (for proportional scheduling): 4
Process 1
Burst Time: 2
Deadline: 4
Period: 5
Weight (for proportional scheduling): 2
Process 2
Burst Time: 2
Deadline: 8
Period: 10
Weight (for proportional scheduling): 4
===== Earliest Deadline First (EDF) =====
CPU Utilization: 1.18
Schedulable: NO
ID BT DL CT WT TAT
1 2 4 2 0 2
0 3 7 5 2 5
2 2 8 7 5 7
===== Rate Monotonic Scheduling (RMS) =====
CPU Utilization: 0.75
RM Bound: 0.7798
Schedulable: YES
ID BT PR CT WT TAT
1 2 5 2 0 2
2 2 10 4 2 4
0 3 20 7 4 7
===== Proportional Scheduling =====
ID Weight CPU Share(%)
1 2 20.00%
2 4 40.00%
0 4 40.00%

LAB 4

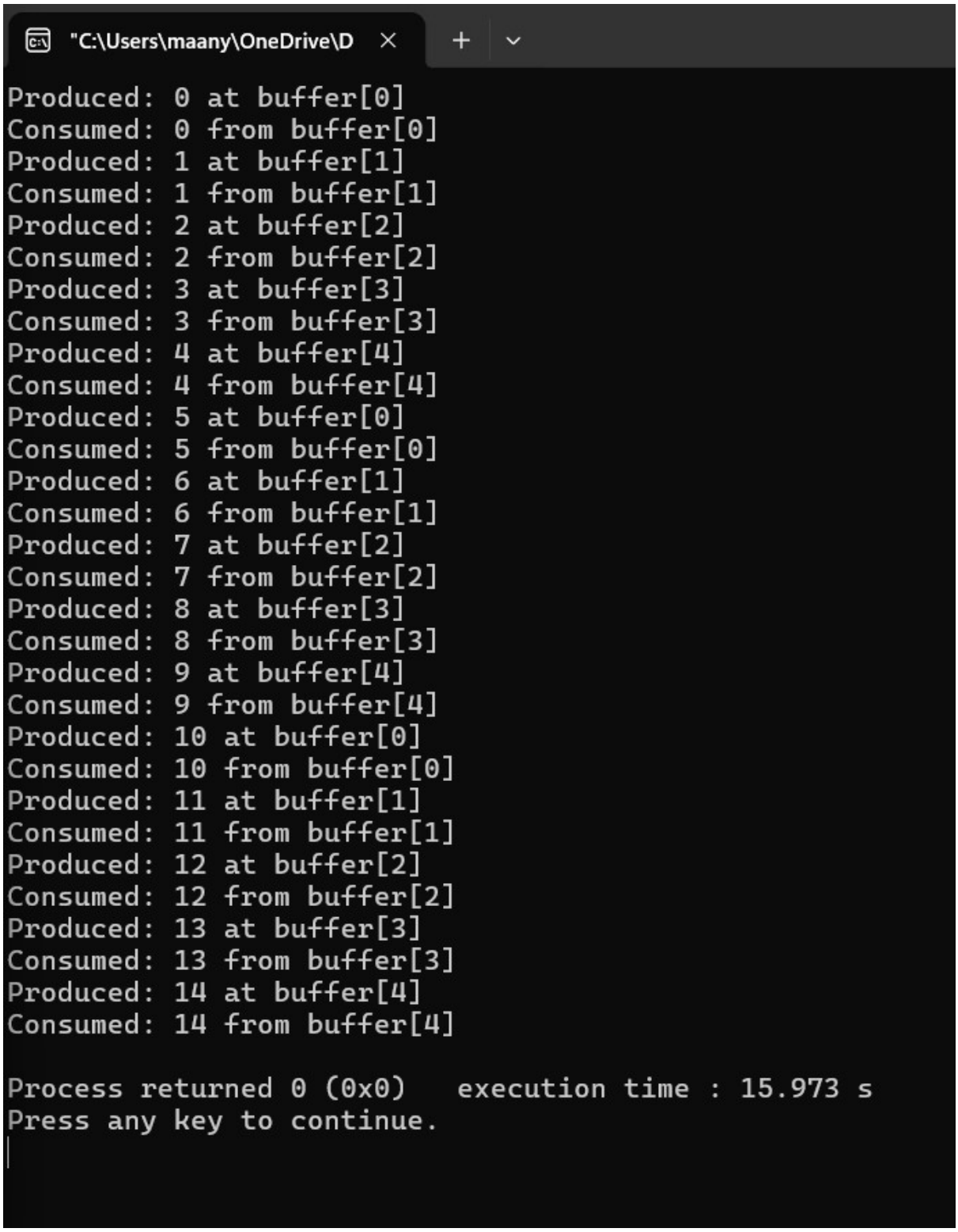
Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem

a) Producer-Consumer problem

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define BUFFER_SIZE 5
#define MAX 15

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full, mutex;

void* producer(void* arg) {
   for (int i = 0; i < MAX; i++) {
      sem_wait(&empty);
      sem_wait(&mutex);
      buffer[in] = i;
      printf("Produced: %d at buffer[%d]\n", i, in);
      in = (in + 1) % BUFFER_SIZE;
      sem_post(&mutex);
      sem_post(&full);
      sleep(1);
   }
   return NULL;
}
```

```
void* consumer(void* arg) {
    for (int i = 0; i < MAX; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&empty);
        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t p, c;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);
    return 0;
}
```

OUTPUT:


"C:\Users\maany\OneDrive\D
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Consumed: 2 from buffer[2]
Produced: 3 at buffer[3]
Consumed: 3 from buffer[3]
Produced: 4 at buffer[4]
Consumed: 4 from buffer[4]
Produced: 5 at buffer[0]
Consumed: 5 from buffer[0]
Produced: 6 at buffer[1]
Consumed: 6 from buffer[1]
Produced: 7 at buffer[2]
Consumed: 7 from buffer[2]
Produced: 8 at buffer[3]
Consumed: 8 from buffer[3]
Produced: 9 at buffer[4]
Consumed: 9 from buffer[4]
Produced: 10 at buffer[0]
Consumed: 10 from buffer[0]
Produced: 11 at buffer[1]
Consumed: 11 from buffer[1]
Produced: 12 at buffer[2]
Consumed: 12 from buffer[2]
Produced: 13 at buffer[3]
Consumed: 13 from buffer[3]
Produced: 14 at buffer[4]
Consumed: 14 from buffer[4]
Process returned 0 (0x0) execution time : 15.973 s
Press any key to continue.

b) Dining-Philosopher's problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5

sem_t forks[N];
sem_t room;

void* philosopher(void* num) {
    int id = *(int*)num;
    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        sem_wait(&room);
        if (id % 2 == 0) {
            sem_wait(&forks[id]);
            printf("Philosopher %d picked up left fork %d.\n", id, id);
            sem_wait(&forks[(id + 1) % N]);
            printf("Philosopher %d picked up right fork %d.\n", id, (id + 1) % N);
        } else {
            sem_wait(&forks[(id + 1) % N]);
            printf("Philosopher %d picked up right fork %d.\n", id, (id + 1) % N);
            sem_wait(&forks[id]);
            printf("Philosopher %d picked up left fork %d.\n", id, id);
        }
        printf("Philosopher %d is eating.\n", id);
        sleep(1);
        sem_post(&forks[id]);
        sem_post(&forks[(id + 1) % N]);
```

```
        printf("Philosopher %d put down forks %d and %d.\n", id, id, (id + 1) % N);
        sem_post(&room);
    }
}

int main() {
    pthread_t phil[N];
    int ids[N];
    sem_init(&room, 0, N - 1);
    for (int i = 0; i < N; i++) {
        sem_init(&forks[i], 0, 1);
    }
    for (int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&phil[i], NULL, philosopher, &ids[i]);
    }
    for (int i = 0; i < N; i++) {
        pthread_join(phil[i], NULL);
    }
    return 0;
}
```

OUTPUT:


"C:\Users\maany\OneDrive\D
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 0 put down forks 0 and 1.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 is thinking.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 2 is thinking.

LAB 5

Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection

a) Bankers' algorithm

```
#include <stdio.h>
#define MAX 10
int n, m;
int Allocation[MAX][MAX];
int Max[MAX][MAX];
int Need[MAX][MAX];
int Available[MAX];

void calculateNeed() {
   int i, j;
   for(i = 0; i < n; i++) {
      for(j = 0; j < m; j++) {
         Need[i][j] = Max[i][j] - Allocation[i][j];
      }
   }
}
int safetyAlgorithm() {
   int Work[MAX];
   int Finish[MAX];
   int SafeSequence[MAX];
   int i, j, k;
   for(i = 0; i < m; i++)
      Work[i] = Available[i];
   for(i = 0; i < n; i++)
      Finish[i] = 0;
   int count = 0;
   while(count < n) {
```

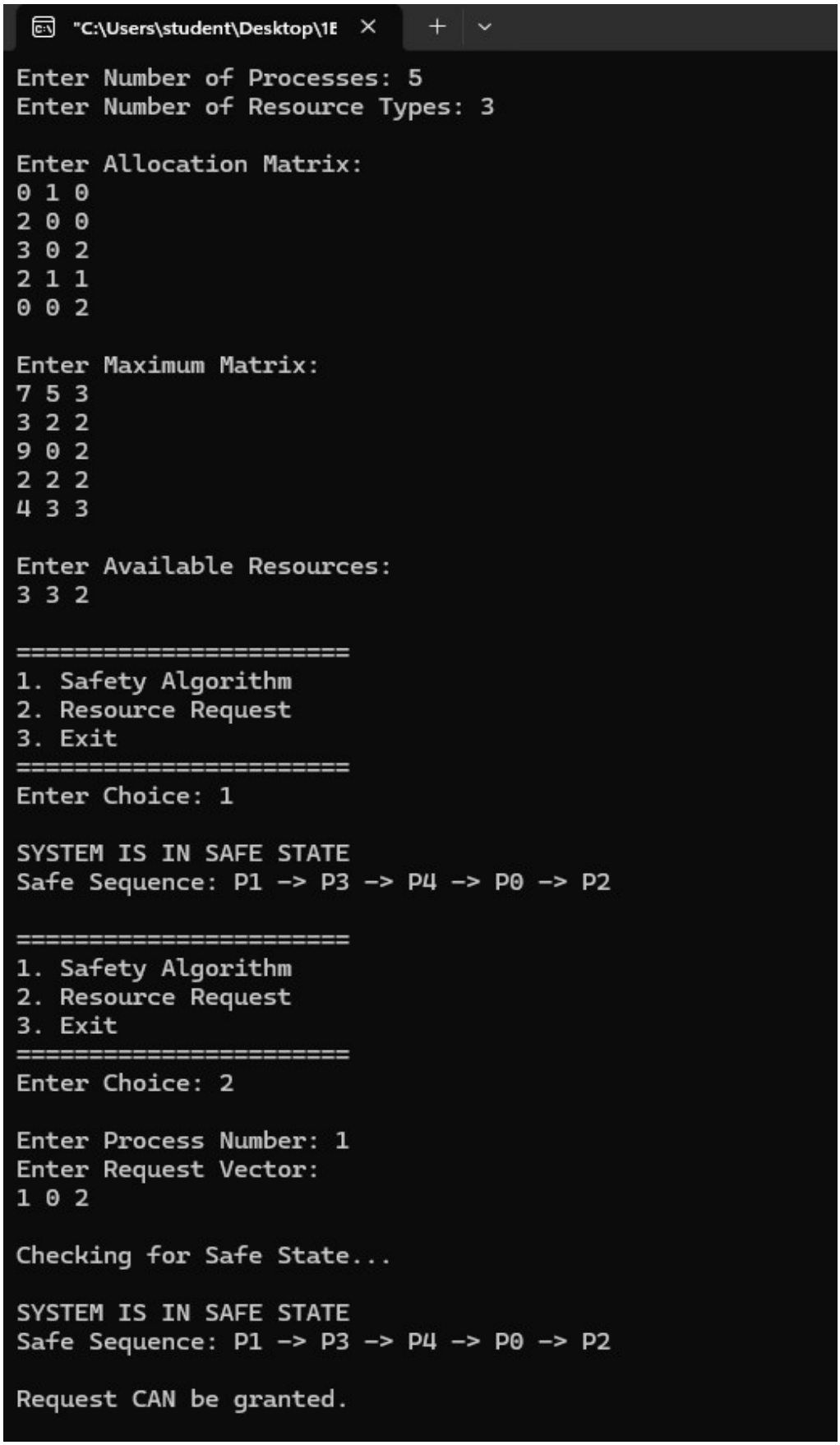
```
    int found = 0;
    for(i = 0; i < n; i++) {
        if(Finish[i] == 0) {
            for(j = 0; j < m; j++) {
                if(Need[i][j] > Work[j])
                    break;
            }
            if(j == m) {
                for(k = 0; k < m; k++)
                    Work[k] += Allocation[i][k];
                SafeSequence[count++] = i;
                Finish[i] = 1;
                found = 1;
            }
        }
    }
    if(found == 0)
        break;
}
if(count == n) {
    printf("\nSYSTEM IS IN SAFE STATE\n");
    printf("Safe Sequence: ");
    for(i = 0; i < n; i++) {
        printf("P%d", SafeSequence[i]);
        if(i != n - 1)
            printf(" -> ");
    }
    printf("\n");
    return 1;
}
else {
```

```
        printf("\nSYSTEM IS IN UNSAFE STATE\n");
        return 0;
    }
}
void resourceRequest() {
    int process;
    int Request[MAX];
    int i;
    printf("\nEnter Process Number: ");
    scanf("%d", &process);
    printf("Enter Request Vector:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &Request[i]);
    for(i = 0; i < m; i++) {
        if(Request[i] > Need[process][i]) {
            printf("\nERROR: Process exceeded maximum claim.\n");
            return;
        }
    }
    for(i = 0; i < m; i++) {
        if(Request[i] > Available[i]) {
            printf("\nResources not available. Process must wait.\n");
            return;
        }
    }
    for(i = 0; i < m; i++) {
        Available[i] -= Request[i];
        Allocation[process][i] += Request[i];
        Need[process][i] -= Request[i];
    }
    printf("\nChecking for Safe State...\n");
```

```
    if(safetyAlgorithm()) {
        printf("\nRequest CAN be granted.\n");
    }
    else {
        for(i = 0; i < m; i++) {
            Available[i] += Request[i];
            Allocation[process][i] -= Request[i];
            Need[process][i] += Request[i];
        }
        printf("\nRequest CANNOT be granted.");
        printf("\nSystem restored to previous state.\n");
    }
}

int main() {
    int i, j;
    int choice;
    printf("Enter Number of Processes: ");
    scanf("%d", &n);
    printf("Enter Number of Resource Types: ");
    scanf("%d", &m);
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &Allocation[i][j]);
        }
    }
    printf("\nEnter Maximum Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &Max[i][j]);
```

```
        }
    }
    printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &Available[i]);
    }
    calculateNeed();
    do {
        printf("\n=======================");
        printf("\n1. Safety Algorithm");
        printf("\n2. Resource Request");
        printf("\n3. Exit");
        printf("\n=======================");
        printf("\nEnter Choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                safetyAlgorithm();
                break;
            case 2:
                resourceRequest();
                break;
            case 3:
                printf("\nExiting...\n");
                break;
            default:
                printf("\nInvalid Choice\n");
        }
    } while(choice != 3);
    return 0;
}
```

OUTPUT:


"C:\Users\student\Desktop\1E
Enter Number of Processes: 5
Enter Number of Resource Types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2
1. Safety Algorithm
2. Resource Request
3. Exit
Enter Choice: 1
SYSTEM IS IN SAFE STATE
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
1. Safety Algorithm
2. Resource Request
3. Exit
Enter Choice: 2
Enter Process Number: 1
Enter Request Vector:
1 0 2
Checking for Safe State...
SYSTEM IS IN SAFE STATE
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
Request CAN be granted.

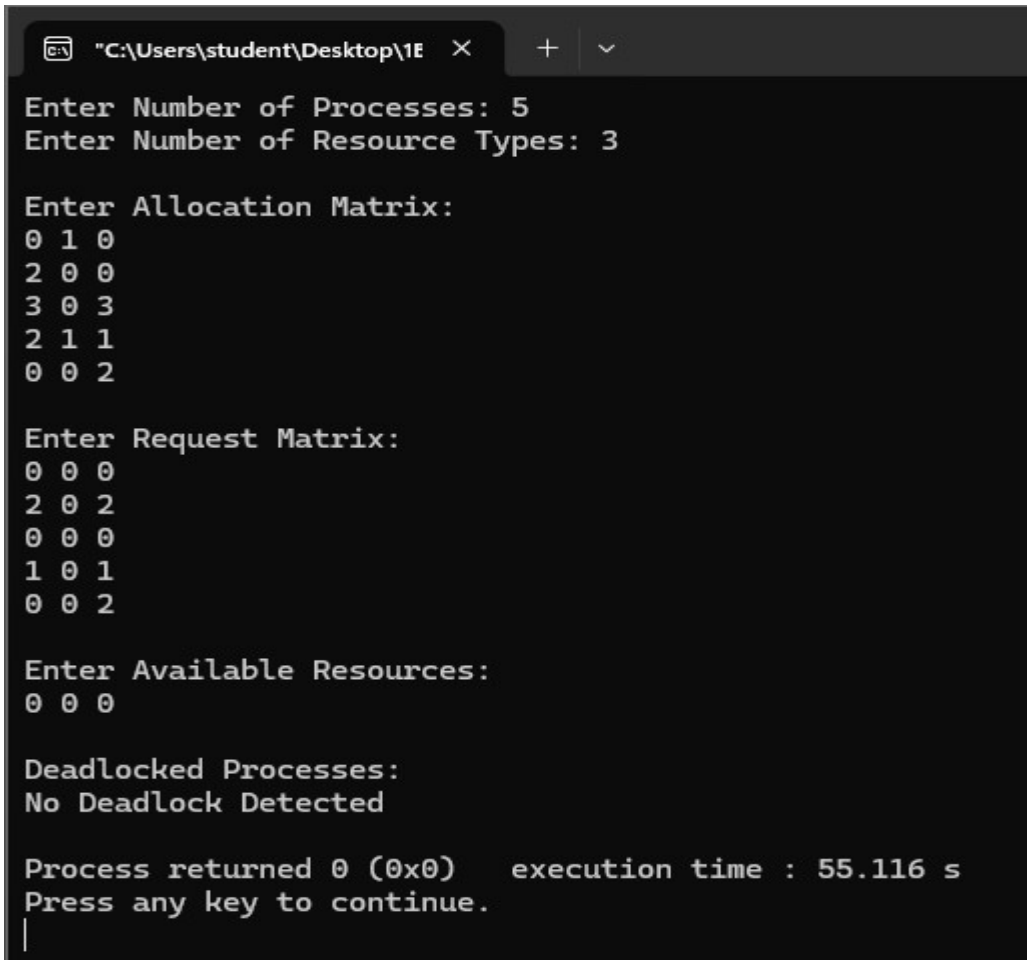
b) Deadlock Detection

```
#include <stdio.h>
#define MAX 10
int main() {
    int n, m;
    int Allocation[MAX][MAX];
    int Request[MAX][MAX];
    int Available[MAX];
    int Work[MAX];
    int Finish[MAX];
    int i, j, k;
    printf("Enter Number of Processes: ");
    scanf("%d", &n);
    printf("Enter Number of Resource Types: ");
    scanf("%d", &m);
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &Allocation[i][j]);
        }
    }
    printf("\nEnter Request Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &Request[i][j]);
        }
    }
    printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &Available[i]);
    }
```

```
for(i = 0; i < m; i++)
    Work[i] = Available[i];
for(i = 0; i < n; i++) {
    int allZero = 1;
    for(j = 0; j < m; j++) {
        if(Allocation[i][j] != 0) {
            allZero = 0;
            break;
        }
    }
    if(allZero)
        Finish[i] = 1;
    else
        Finish[i] = 0;
}
int found;
do {
    found = 0;
    for(i = 0; i < n; i++) {
        if(Finish[i] == 0) {
            for(j = 0; j < m; j++) {
                if(Request[i][j] > Work[j])
                    break;
            }
            if(j == m) {
                for(k = 0; k < m; k++)
                    Work[k] += Allocation[i][k];
                Finish[i] = 1;
                found = 1;
            }
        }
```

```
        }
    } while(found);
    int deadlock = 0;
    printf("\nDeadlocked Processes:\n");
    for(i = 0; i < n; i++) {
        if(Finish[i] == 0) {
            printf("P%d\n", i);
            deadlock = 1;
        }
    }
    if(deadlock == 0)
        printf("No Deadlock Detected\n");
    return 0;
}
```

OUTPUT:



LAB 6

Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit

```
#include <stdio.h>
void firstFit(int blockSize[], int m, int processSize[], int n)
{
   int allocation[n];
   for (int i = 0; i < n; i++)
      allocation[i] = -1;
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < m; j++)
      {
         if (blockSize[j] >= processSize[i])
         {
            allocation[i] = j;
            blockSize[j] -= processSize[i];
            break;
         }
      }
   }
   printf("\n--- First Fit ---\n");
   printf("Process No.\tProcess Size\tBlock No.\n");
   for (int i = 0; i < n; i++)
   {
      printf("%d\t\t%d\t\t", i + 1, processSize[i]);

      if (allocation[i] != -1)
         printf("%d\n", allocation[i] + 1);
      else
         printf("Not Allocated\n");
```

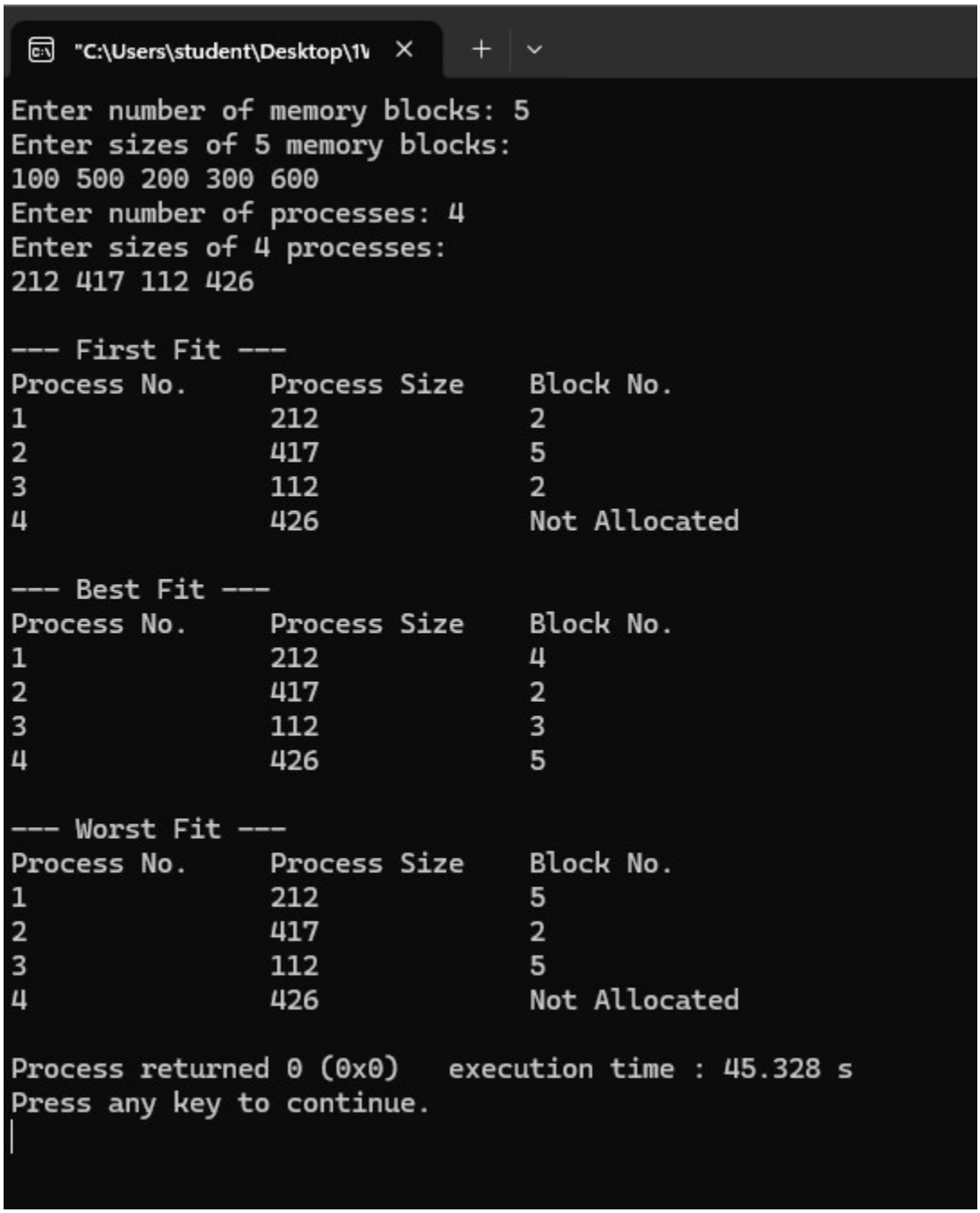
```
    }
}

void bestFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++)
    {
        int bestIdx = -1;
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                    bestIdx = j;
            }
        }
        if (bestIdx != -1)
        {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
    }
    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
```

```
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++)
    {
        int worstIdx = -1;
        for (int j = 0; j < m; j++)
        {
            if (blockSize[j] >= processSize[i])
            {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }
        if (worstIdx != -1)
        {
            allocation[i] = worstIdx;
            blockSize[worstIdx] -= processSize[i];
        }
    }
    printf("\n--- Worst Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++)
```

```
    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

int main()
{
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blockSize[m];
    int block1[m], block2[m], block3[m];
    printf("Enter sizes of %d memory blocks:\n", m);
    for (int i = 0; i < m; i++)
    {
        scanf("%d", &blockSize[i]);
        block1[i] = blockSize[i];
        block2[i] = blockSize[i];
        block3[i] = blockSize[i];
    }
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int processSize[n];
    printf("Enter sizes of %d processes:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &processSize[i]);
```

```
    firstFit(block1, m, processSize, n);
    bestFit(block2, m, processSize, n);
    worstFit(block3, m, processSize, n);
    return 0;
}
```

OUTPUT:



LAB 7

Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal

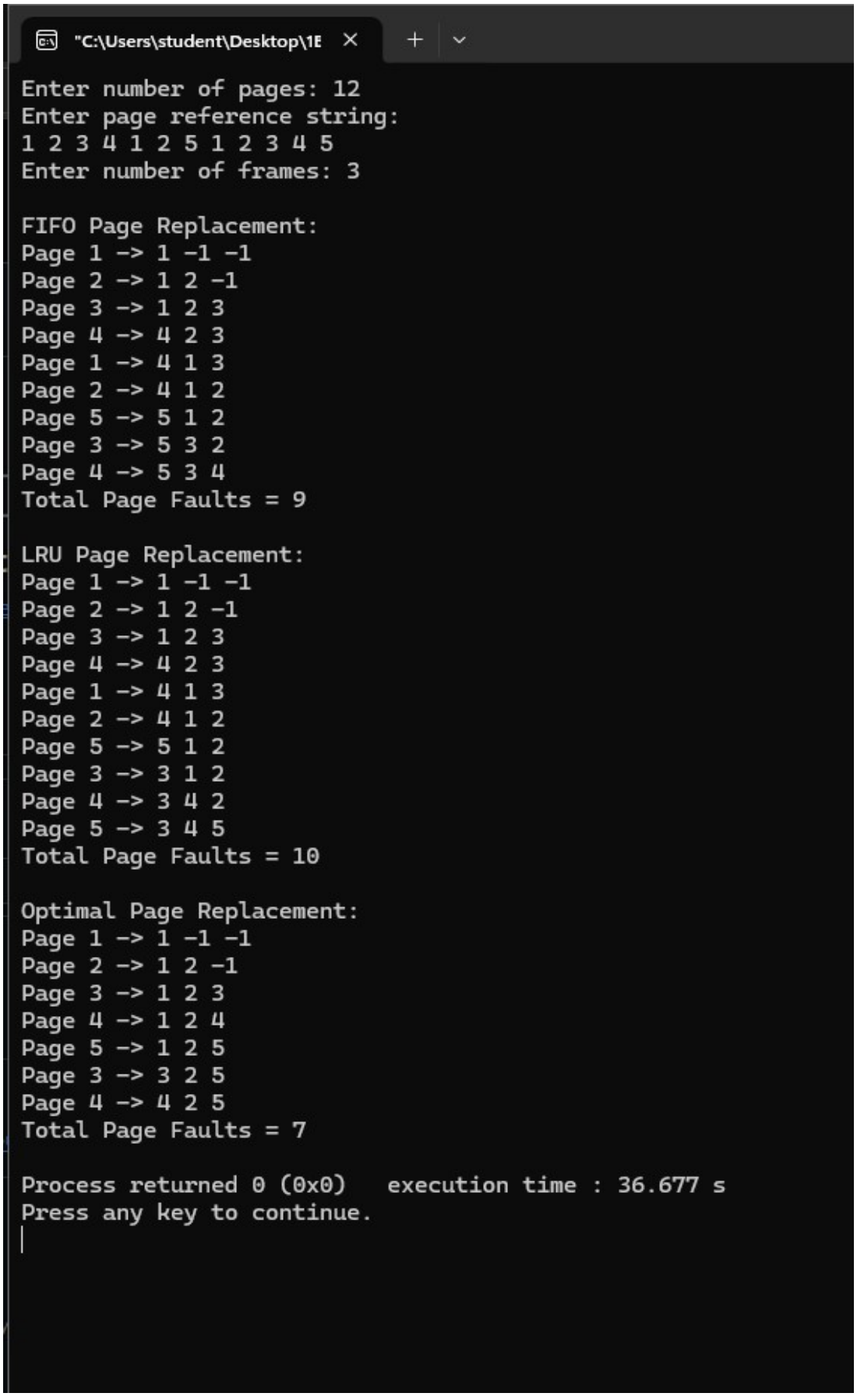
```
#include <stdio.h>
#define MAX 50

int search(int key, int frame[], int frameSize) {
    for (int i = 0; i < frameSize; i++) {
        if (frame[i] == key)
            return i;
    }
    return -1;
}

void FIFO(int pages[], int n, int frameSize) {
    int frame[MAX], faults = 0, index = 0;
    for (int i = 0; i < frameSize; i++)
        frame[i] = -1;
    printf("\nFIFO Page Replacement:\n");
    for (int i = 0; i < n; i++) {
        if (search(pages[i], frame, frameSize) == -1) {
            frame[index] = pages[i];
            index = (index + 1) % frameSize;
            faults++;
            printf("Page %d -> ", pages[i]);
            for (int j = 0; j < frameSize; j++)
                printf("%d ", frame[j]);
            printf("\n");
        }
    }
    printf("Total Page Faults = %d\n", faults);
```

```
}

void LRU(int pages[], int n, int frameSize) {
    int frame[MAX], time[MAX];
    int faults = 0, counter = 0;
    for (int i = 0; i < frameSize; i++) {
        frame[i] = -1;
        time[i] = 0;
    }
    printf("\nLRU Page Replacement:\n");
    for (int i = 0; i < n; i++) {
        int pos = search(pages[i], frame, frameSize);
        if (pos != -1) {
            time[pos] = ++counter;
        } else {
            int lru = 0;
            for (int j = 1; j < frameSize; j++) {
                if (time[j] < time[lru])
                    lru = j;
            }

            frame[lru] = pages[i];
            time[lru] = ++counter;
            faults++;
            printf("Page %d -> ", pages[i]);
            for (int j = 0; j < frameSize; j++)
                printf("%d ", frame[j]);
            printf("\n");
        }
    }
    printf("Total Page Faults = %d\n", faults);
```

```
}

int predict(int pages[], int frame[], int n, int index, int frameSize) {
    int farthest = index, pos = -1;
    for (int i = 0; i < frameSize; i++) {
        int j;
        for (j = index; j < n; j++) {
            if (frame[i] == pages[j]) {
                if (j > farthest) {
                    farthest = j;
                    pos = i;
                }
                break;
            }
        }
        if (j == n)
            return i;
    }
    return (pos == -1) ? 0 : pos;
}

void Optimal(int pages[], int n, int frameSize) {
    int frame[MAX];
    int faults = 0;
    int filled = 0;
    for (int i = 0; i < frameSize; i++)
        frame[i] = -1;
    printf("\nOptimal Page Replacement:\n");
    for (int i = 0; i < n; i++) {
        if (search(pages[i], frame, frameSize) == -1) {
            if (filled < frameSize) {
```

```
            frame[filled++] = pages[i];
        } else {
            int pos = predict(pages, frame, n, i + 1, frameSize);
            frame[pos] = pages[i];
        }
        faults++;
        printf("Page %d -> ", pages[i]);
        for (int j = 0; j < frameSize; j++)
            printf("%d ", frame[j]);
        printf("\n");
      }
   }
   printf("Total Page Faults = %d\n", faults);
}

int main() {
   int pages[MAX], n, frameSize;
   printf("Enter number of pages: ");
   scanf("%d", &n);
   printf("Enter page reference string:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &pages[i]);
   printf("Enter number of frames: ");
   scanf("%d", &frameSize);
   FIFO(pages, n, frameSize);
   LRU(pages, n, frameSize);
   Optimal(pages, n, frameSize);
   return 0;
}
```

OUTPUT:


"C:\Users\student\Desktop\1E
Enter number of pages: 12
Enter page reference string:
1 2 3 4 1 2 5 1 2 3 4 5
Enter number of frames: 3
FIFO Page Replacement:
Page 1 -> 1 -1 -1
Page 2 -> 1 2 -1
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 4 1 3
Page 2 -> 4 1 2
Page 5 -> 5 1 2
Page 3 -> 5 3 2
Page 4 -> 5 3 4
Total Page Faults = 9
LRU Page Replacement:
Page 1 -> 1 -1 -1
Page 2 -> 1 2 -1
Page 3 -> 1 2 3
Page 4 -> 4 2 3
Page 1 -> 4 1 3
Page 2 -> 4 1 2
Page 5 -> 5 1 2
Page 3 -> 3 1 2
Page 4 -> 3 4 2
Page 5 -> 3 4 5
Total Page Faults = 10
Optimal Page Replacement:
Page 1 -> 1 -1 -1
Page 2 -> 1 2 -1
Page 3 -> 1 2 3
Page 4 -> 1 2 4
Page 5 -> 1 2 5
Page 3 -> 3 2 5
Page 4 -> 4 2 5
Total Page Faults = 7
Process returned 0 (0x0) execution time : 36.677 s
Press any key to continue.